Day 69 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 10, Day 7 reference

Watch: Sequence-to-Sequence (seq2seq) Encoder-Decoder Neural Networks, Clearly Explained!!! (StatQuest) | Paper Review: Sequence to Sequence Learning with Neural Networks (Aladdin Persson)

`day69_seq2seq_encoder_decoder.py`, segment by segment, in the order `main()` uses things. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula; the green **SYSTEM DIAGRAM** and the graph are the ones the script saved. A glossary is at the end.

MATH BOX LEGEND

Symbols: $x_{1..n}$ = source digits · $y_{1..n}$ = target digits · (h_{enc}, c_{enc}) = the encoder's final states · GO = 9 starts decoding, EOS = 0 ends it · V = 10 tokens

Segment 0 — Setup

Imports and the token set

CODE

```
import time
import numpy as np
from threadpoolctl import threadpool_limits
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
np.random.seed(42)
EOS, GO = 0, 9
V = 10
LENGTHS = range(3, 9)
SEEDS = [0, 1, 2]
STEPS = 20000
EVERY = 1000
```

- `EOS, GO = 0, 9` — two special tokens: 0 ends a sequence, 9 tells the decoder to start writing.
- `LENGTHS = range(3, 9)` — a range object: 3, 4, ..., 8 (the end is excluded).
- `STEPS, EVERY` — training steps per run, and how often test accuracy is measured.

Segment 1 — The reused parts

Parts 1-2: one LSTM step and the loop (Day 63-65 exercises, verbatim)

CODE

```
def lstm_step_forward(x_t, h_prev, c_prev, W, b):
    H = h_prev.shape[1]
    a = np.concatenate([x_t, h_prev], axis=1) @ W + b
    f = sigmoid(a[:, 0 * H:1 * H])
    i = sigmoid(a[:, 1 * H:2 * H])
    g = np.tanh(a[:, 2 * H:3 * H])
    o = sigmoid(a[:, 3 * H:4 * H])
    c = f * c_prev + i * g
    h = o * np.tanh(c)
    return h, c, (f, i, g, o)
def lstm_forward(X, h0, c0, W, b):
    m, T, _ = X.shape
    H = h0.shape[1]
    Hs, Cs = np.zeros((m, T, H)), np.zeros((m, T, H))
    gates = []
    h, c = h0, c0
    for t in range(T):
        h, c, fiog = lstm_step_forward(X[:, t, :], h, c, W, b)
        Hs[:, t, :], Cs[:, t, :] = h, c
        gates.append(fiog)
    return Hs, Cs, gates
```

- `h, c = h0, c0` — the loop starts from whatever states it is given: zeros for the encoder, the encoder's final states for the decoder.
- `gates.append(fiog)` — every step's (f, i, g, o), kept for the backward pass.

Part 3: the backward pass, with three changed lines

MATH

after the last backward step: $dh = dL/dh_0$, $dc = dL/dc_0$; *for the decoder,* $h_0 = h_{enc}$ *and* $c_0 = c_{enc}$

CODE

```
def lstm_backward(dHs, dh_last, dc_last, X, h0, c0, W, Hs, Cs, gates):
    m, T, D = X.shape
    dW, db = np.zeros_like(W), np.zeros(W.shape[1])
    dh, dc = dh_last.copy(), dc_last.copy()
    for t in reversed(range(T)):
        dh = dh + dHs[:, t, :]
        f, i, g, o = gates[t]
        h_prev = Hs[:, t - 1, :] if t > 0 else h0
        c_prev = Cs[:, t - 1, :] if t > 0 else c0
        tanh_c = np.tanh(Cs[:, t, :])
        do = dh * tanh_c
        dc = dc + dh * o * (1 - tanh_c ** 2)
        da = np.concatenate([dc * c_prev * f * (1 - f),
                             dc * g * i * (1 - i),
                             dc * i * (1 - g ** 2),
                             do * o * (1 - o)], axis=1)
        z = np.concatenate([X[:, t, :], h_prev], axis=1)
        dW += z.T @ da
        db += da.sum(axis=0)
        dh = (da @ W.T)[:, D:]
        dc = dc * f
    return dW, db, dh, dc
```

- `dh, dc = dh_last.copy(), dc_last.copy()` — CHANGED: the encoder's backward pass starts with gradient at its last h AND its last c.
- `dh = dh + dHs[:, t, :]` — Day 67's change: the decoder's output at step t adds its gradient here.

- `return dW, db, dh, dc` — CHANGED: after the loop, dh and dc are the gradients at h0 and c0, which the encoder needs.

Part 4: a loss at every output step (Day 67 exercise)

MATH

$$L = -(1/mT)\,\Sigma_t\,\Sigma_j \log p_t(y_{j,t}); \quad d\,logits_t = (p_t - onehot(y_t))/mT$$

CODE

```
def outputs_loss(Hs, Y, Why, by):
    m, T = Y.shape
    loss, dHs = 0.0, np.zeros_like(Hs)
    dWhy, dby = np.zeros_like(Why), np.zeros_like(by)
    for t in range(T):
        p = softmax(Hs[:, t] @ Why + by)
        loss -= np.log(p[np.arange(m), Y[:, t]]).sum() / (m * T)
        dlog = p.copy(); dlog[np.arange(m), Y[:, t]] -= 1; dlog /= m * T
        dWhy += Hs[:, t].T @ dlog; dby += dlog.sum(axis=0)
        dHs[:, t] = dlog @ Why.T
    return loss, dHs, dWhy, dby
```

- `dlog /= m * T` — the loss is averaged over the batch AND the steps, so the gradient is too.
- `dHs[:, t] = dlog @ Why.T` — what step t's output sends back to that step's hidden state.

Segment 2 — The assembled model

Seq2Seq.__init__ and encode

CODE

```
class Seq2Seq:
    def __init__(self, hidden=32, bidirectional=False, seed=0, forget_bias=1.0, lr=0.5, clip=1.0):
        rng = np.random.RandomState(seed)
        self.H, self.bi, self.lr, self.clip = hidden, bidirectional, lr, clip
        He = hidden // 2 if bidirectional else hidden
        self.P = {"Wef": rng.randn(V + He, 4 * He) / np.sqrt(V + He), "bef": np.zeros(4 * He)}
        if bidirectional:
            self.P.update(Web=rng.randn(V + He, 4 * He) / np.sqrt(V + He), beb=np.zeros(4 * He))
        self.P.update(Wd=rng.randn(V + hidden, 4 * hidden) / np.sqrt(V + hidden), bd=np.zeros(4 *
            hidden),
                      Why=rng.randn(hidden, V) / np.sqrt(hidden), by=np.zeros(V))
        for name, size in [("bef", He), ("beb", He), ("bd", hidden)]:
            if name in self.P:
                self.P[name][:size] = forget_bias
    def encode(self, Xs):
        m = Xs.shape[0]
        He = self.P["Wef"].shape[1] // 4
        z = np.zeros((m, He))
        Hf, Cf, gf = lstm_forward(Xs, z, z, self.P["Wef"], self.P["bef"])
        caches = [("f", Xs, Hf, Cf, gf)]
        h, c = Hf[:, -1], Cf[:, -1]
        if self.bi:
            Xr = Xs[:, ::-1]
            Hr, Cr, gr = lstm_forward(Xr, z, z, self.P["Web"], self.P["beb"])
            caches.append(("b", Xr, Hr, Cr, gr))
            h = np.concatenate([h, Hr[:, -1]], axis=1)
            c = np.concatenate([c, Cr[:, -1]], axis=1)
        return h, c, caches
```

- `He = hidden // 2 if bidirectional else hidden` — with two encoder directions, each gets half, so the decoder is 32 either way.
- `self.P = {...}` — all parameters in one dictionary: training loops over its keys.

- `Xs[:, ::-1]` — every sequence reversed in one slice; all sequences in a batch have the same length, so no padding is involved.
- `Hf[:, -1], Cf[:, -1]` — the encoder's FINAL states: the only thing passed to the decoder.
- `np.concatenate([h, Hr[:, -1]], axis=1)` — Day 68's join, applied to final states only.

loss_and_grads: through the decoder, then into the encoder

MATH

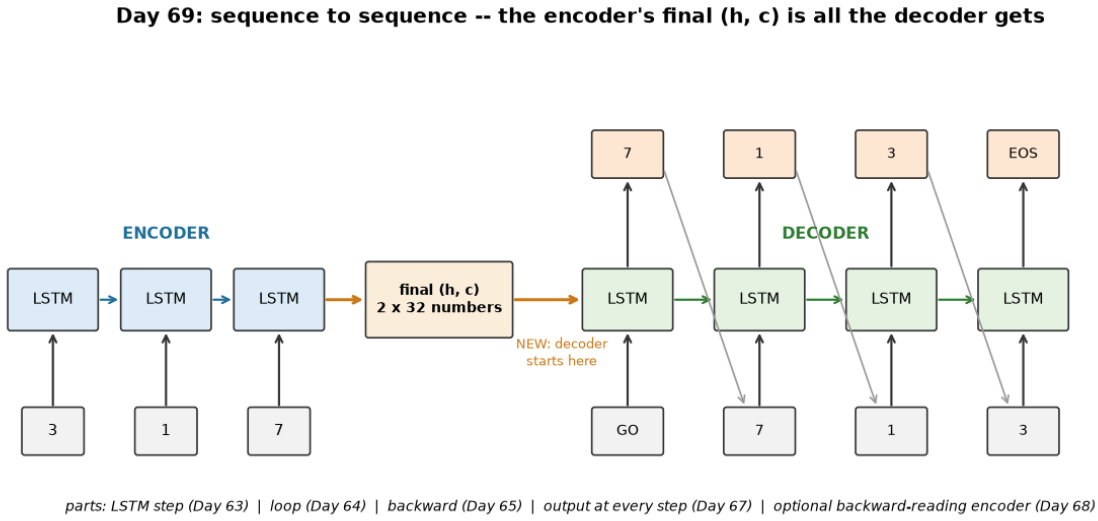
dL/dh_{enc} = (decoder's dh_0), dL/dc_{enc} = (decoder's dc_0) → *start of the encoder's backward pass*

CODE

```
def loss_and_grads(self, Xs, Xd, Y):
    h0, c0, caches = self.encode(Xs)
    Hd, Cd, gd = lstm_forward(Xd, h0, c0, self.P["Wd"], self.P["bd"])
    loss, dHs, dWhy, dby = outputs_loss(Hd, Y, self.P["Why"], self.P["by"])
    g = {"Why": dWhy, "by": dby}
    z = np.zeros_like(h0)
    g["Wd"], g["bd"], dh0, dc0 = lstm_backward(dHs, z, z, Xd, h0, c0, self.P["Wd"], Hd, Cd, gd)
    He = self.P["Wef"].shape[1] // 4
    for j, (d, X, Hs, Cs, gates) in enumerate(caches):
        part = slice(j * He, (j + 1) * He)
        ze = np.zeros((X.shape[0], He))
        g["We" + d], g["be" + d], _, _ = lstm_backward(np.zeros_like(Hs), dh0[:, part], dc0[:,
            part],
                                                       X, ze, ze, self.P["We" + d], Hs, Cs, gates)
    return loss, g
```

- `lstm_forward(Xd, h0, c0, ...)` — NEW: the decoder starts from the encoder's states.
- `g["Wd"], g["bd"], dh0, dc0 = lstm_backward(...)` — the decoder's backward pass also returns the gradient at its starting states.
- `part = slice(j * He, (j + 1) * He)` — a slice object: which columns of dh0 and dc0 belong to this encoder direction.
- `lstm_backward(np.zeros_like(Hs), dh0[:, part], dc0[:, part], ...)` — the encoder's outputs had no loss of their own; gradient enters only at its last step.
- `"We" + d` — string concatenation builds the key: Wef or Web.

SYSTEM DIAGRAM



The encoder reads 3 1 7; its final (h, c) starts the decoder, which writes 7 1 3 EOS.

train_step, generate and teacher_forced

CODE

```
def train_step(self, Xs, Xd, Y):
    loss, g = self.loss_and_grads(Xs, Xd, Y)
    norm = np.sqrt(sum(np.sum(v ** 2) for v in g.values()))
    scale = min(1.0, self.clip / norm)
    for k in self.P:
        self.P[k] -= self.lr * scale * g[k]
    return loss
def generate(self, Xs, n_steps, zero_state=False):
    h, c, _ = self.encode(Xs)
    if zero_state:
        h, c = np.zeros_like(h), np.zeros_like(c)
    token = np.full(len(Xs), GO)
    out = []
    for _ in range(n_steps):
        h, c, _ = lstm_step_forward(np.eye(V)[token], h, c, self.P["Wd"], self.P["bd"])
        token = (h @ self.P["Why"] + self.P["by"]).argmax(axis=1)
        out.append(token)
    return np.stack(out, axis=1)
def teacher_forced(self, Xs, Xd):
    h0, c0, _ = self.encode(Xs)
    Hd, _, _ = lstm_forward(Xd, h0, c0, self.P["Wd"], self.P["bd"])
    return (Hd @ self.P["Why"] + self.P["by"]).argmax(axis=2)
```

- `scale = min(1.0, self.clip / norm)` — Day 60's clipping in one line: shrink only when the norm is above the threshold.
- `token = np.full(len(Xs), GO)` — an array filled with one value: every sequence starts with GO.
- `np.eye(V)[token]` — rows of the identity matrix are one-hot vectors; indexing with the tokens picks them.
- `out.append(token)` — the chosen token is kept and becomes the next step's input: greedy decoding.
- `argmax(axis=2)` — with (m, T, V) logits, axis 2 is the vocabulary.

Segment 3 — Data, training and evaluation

make_batch and evaluate

CODE

```
def make_batch(rng, m, n, task):
    src = rng.randint(1, 9, size=(m, n))
    tgt = src if task == "copy" else src[:, ::-1]
    dec_in = np.concatenate([np.full((m, 1), GO), tgt], axis=1)
    dec_out = np.concatenate([tgt, np.full((m, 1), EOS)], axis=1)
    return np.eye(V)[src], np.eye(V)[dec_in], dec_out, src
def test_sets(task, n_per_length=300):
    rng = np.random.RandomState(12345)
    return {n: make_batch(rng, n_per_length, n, task) for n in LENGTHS}
def evaluate(net, tests, zero_state=False):
    exact, tokens = [], []
    for n, (Xs, _, Y, _) in tests.items():
        pred = net.generate(Xs, n + 1, zero_state)
        exact.append(np.all(pred == Y, axis=1).mean())
        tokens.append((pred == Y).mean())
    return np.mean(exact), np.mean(tokens), exact
```

- `tgt = src if task == "copy" else src[:, ::-1]` — the only difference between the two tasks.
- `np.concatenate([np.full((m, 1), GO), tgt], axis=1)` — the decoder reads GO and then the target shifted one step right (teacher forcing).
- `{n: make_batch(...) for n in LENGTHS}` — a dictionary comprehension: one fixed test batch per length.
- `np.all(pred == Y, axis=1)` — True only where every token of a sequence is right: exact match.

train and train_cached

CODE

```
def train(task, bidirectional, seed, steps=STEPS, every=EVERY):
    net = Seq2Seq(bidirectional=bidirectional, seed=seed)
    rng = np.random.RandomState(seed + 100)
    tests = test_sets(task)
    curve, losses = [], []
    start = time.perf_counter()
    for step in range(1, steps + 1):
        n = rng.randint(3, 9)
        Xs, Xd, Y, _ = make_batch(rng, 32, n, task)
        losses.append(net.train_step(Xs, Xd, Y))
        if step % every == 0:
            curve.append(evaluate(net, tests)[0])
    return net, np.array(curve), np.mean(losses[-every:]), time.perf_counter() - start
RUNS = {}
def train_cached(task, bidirectional, seed):
    key = (task, bidirectional, seed)
    if key not in RUNS:
        RUNS[key] = train(task, bidirectional, seed)
    return RUNS[key]
```

- `n = rng.randint(3, 9)` — each batch has its own random length, 3 to 8.
- `losses[-every:]` — the last EVERY losses: the mean over recent steps, less noisy than one batch.
- `RUNS[key]` — each (task, encoder, seed) is trained once and reused by every section.

Segment 4 — Analysis

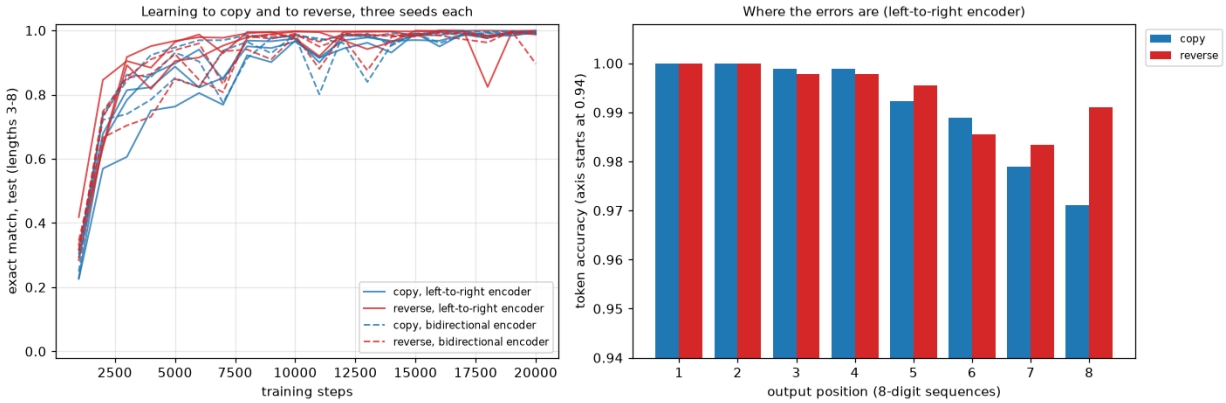
Where the errors are

CODE

```
Xs, _, Y, src = test_sets(task)[8]
acc[task] = np.mean([(train_cached(task, False, s)[0].generate(Xs, 9)[:, :8] == Y[:,
    :8]).mean(axis=0)
                     for s in SEEDS], axis=0)
```

- `test_sets(task)[8]` — the 300 test sequences of length 8.
- `generate(Xs, 9)[:, :8]` — decode 9 tokens (8 digits + EOS) and keep the digits.
- `.mean(axis=0)` — accuracy per output position, averaged over the sequences.

REAL GRAPH



Real graph from the script: learning curves, and accuracy at each output position.

Quick Syntax Glossary — new or reinforced constructs

- `x[:, ::-1]` — reverse along the second axis.
- `np.eye(V)[ids]` — one-hot vectors for an array of token ids.
- `np.full(shape, value)` — an array filled with one value.
- `slice(a, b)` — a reusable slice object; x[:, slice(a, b)] is x[:, a:b].
- `np.all(a, axis=1)` — True where every entry along the axis is True.
- `np.stack(list, axis=1)` — join equal-shaped arrays along a NEW axis.
- `min(1.0, c / norm)` — a clipping factor that is never above 1.
- `range(3, 9)` — 3, 4, 5, 6, 7, 8.

End of Day 69 syntax reference. For the experiments and what they showed, see the Day 69 complete guide.